



University of Puerto Rico



Mayaguez Campus

Department of Electrical and Computer Engineering

Sofia A. Chamorro Messa
Luis
Andrés E. Muñoz Ríos
Sergio Paulo Ruben Avila-Concepción

Prof. José Fernando Vega Riveros
ICOM5015-001D
27 de abril del 2026

1. Purpose

El objetivo de esta asignación es crear y evaluar distintas variantes de búsqueda local. Estaremos investigando y comparando cuatro tipos de búsqueda local en los problemas de 8 reinas y 8 rompecabezas. Estas son el ascenso de colina de ascenso más pronunciado, ascenso de colina de primera elección, ascenso de colina con reinicio aleatorio y recorrido simulado. El rendimiento de cada búsqueda local estará basado en la tasa de éxito en cada instancia y el coste de búsqueda con el número de nodos que genera la búsqueda. Queremos medir cuán robusto son cada una de las búsquedas, su eficiencia y llegar a nuestras propias conclusiones sobre los tópicos teóricos que se estarán experimentando en esta asignación. [1]

2. Key question or hypothesis

El ascenso a colina de ascenso pronunciado tendrá una tasa de éxito baja y costo de búsqueda bajo debido a que suele terminar al verificar que todos los posibles próximos estados son peores que el estado actual. No siempre el estado actual es la solución o meta y la búsqueda es limitada para cuando hay una mejor opción que el estado actual. La búsqueda de ascenso de colina de primera elección es muy similar a la búsqueda de ascenso pronunciado, por lo cual, tendrá resultados similares. En vez de iterar por todos los posibles próximos estados, genera uno al azar y se mueve a ese estado si es mejor. Sin embargo, no verifica todos los posibles estados como en la primera búsqueda. Por otro lado, la búsqueda de ascenso de colina con reinicio aleatorio pensamos que va a

tener la mejor tasa de éxito pero el peor costo de búsqueda debido a que es complicado porque se basa en probabilidad y el costo de búsqueda se acumula por cada instancia. Por último, la búsqueda de recorrido simulado tiene una eficiencia proporcional, es el mejor pero tampoco el peor en ambos criterios. Dependiendo de una diferencia de valor de heurística él determina si es adecuado tomar el mejor próximo estado o si debería tomar el peor próximo estado. Sin embargo, su tasa de éxito debe ser mejor que la búsqueda de ascenso pronunciado porque evita los máximos locales y el costo de búsqueda es mejor que la búsqueda de ascenso de colina con reinicio aleatorio. [1]

3. Concepts

The theoretical framework of this assignment is rooted in the field of metaheuristics and local search algorithms. Unlike classical search algorithms that explore state-space trees to find a path to a goal, local search algorithms operate using a single current node and move only to neighbors of that state [1].

A. Local Search Algorithms

1. Hill-Climbing Search (Steepest-Ascent): This is a loop that continually moves in the direction of increasing value—that is, uphill. It terminates when it reaches a "peak" where no neighbor has a higher value. It is often described as a greedy local search because it grabs the good neighbor state without thinking about where to go next .
2. First-Choice Hill-Climbing: A variant of the steepest-ascent

algorithm that implements a stochastic strategy by generating successors randomly until one is generated that is better than the current state.

3. Hill-Climbing with Random Restart: This meta-algorithm conducts a series of hill-climbing searches from randomly generated initial states, stopping when a goal is found. Its success is based on the probability p of finding a goal from any given starting point, requiring $1/p$ restarts on average.
4. Simulated Annealing: Inspired by the process of annealing in metallurgy, this algorithm shakes the search space to escape local minima by allowing "bad moves" with a probability that decreases over time (the temperature schedule). The probability is governed by the Boltzmann distribution $P = e^{\Delta E/T}$.

B. Problem Domains

1. 8-Puzzle: A sliding-block puzzle consisting of a 3x3 grid with eight numbered tiles and a blank space. The objective is to reach a goal configuration through a sequence of moves. For local search, the Manhattan Distance is utilized as a heuristic function to estimate the value of a state [2].
2. 8-Queens: A constraint satisfaction problem where eight queens must be placed on an 8x8 chessboard such that no two queens threaten each other. The objective function for local search is typically the number of conflicts (pairs of queens attacking each other).

4. Information gathered to be able to carry out the assignment

La referencia teórica principal fue el Capítulo 4 de *Artificial Intelligence: A Modern Approach* de Russell y Norvig [1], que provee pseudocódigo, definiciones formales y valores de referencia para los cuatro algoritmos. El texto reporta que el ascenso de colina de mayor pendiente resuelve aproximadamente el 14% de las instancias de 8 reinas, valor utilizado para validar nuestra implementación.

El repositorio *aima-python* de UC Berkeley sirvió como base de código, proporcionando implementaciones de `hill_climbing`, `simulated_annealing`, `EightPuzzle`, `NQueensProblem` e `InstrumentedProblem`. La función `astar_search` se utilizó para calcular el costo óptimo de solución de cada instancia del 8-puzzle, el cual sirve como eje X en las gráficas de rendimiento.

Para el schedule de enfriamiento del temple simulado, el artículo original de Kirkpatrick, Gelatt y Vecchi (1983) informó la selección del enfriamiento exponencial ($T \leftarrow \alpha \cdot T$) y el rango de temperaturas iniciales. En cuanto a las heurísticas, se seleccionó la distancia Manhattan para el 8-puzzle por su admisibilidad, y la heurística de conteo de conflictos para las 8 reinas ya que mide directamente las violaciones de restricciones.

5. Experiments set up (Methods and tools)

La evaluación experimental se realizó utilizando Python y el repositorio AIMA

(Inteligencia Artificial: Un Enfoque Moderno)[2]. Los experimentos se ejecutaron en un entorno Jupyter Notebook. Para el problema del rompecabezas de 8 piezas, se generaron instancias aplicando movimientos válidos aleatorios desde el estado objetivo. El costo de la solución óptima se calculó mediante la búsqueda A* con la heurística de distancia de Manhattan. Para el problema de las 8 reinas, las configuraciones iniciales se generaron aleatoriamente y se evaluaron en función del número de conflictos.

Los algoritmos evaluados fueron ascenso de colina de máxima pendiente, ascenso de colina de primera elección, ascenso de colina con reinicio aleatorio y recocido simulado. Cada algoritmo se probó en múltiples instancias, y las métricas de rendimiento incluyeron la tasa de éxito y el costo de búsqueda. Se utilizó la clase InstrumentedProblem para medir las expansiones de nodos de forma consistente.

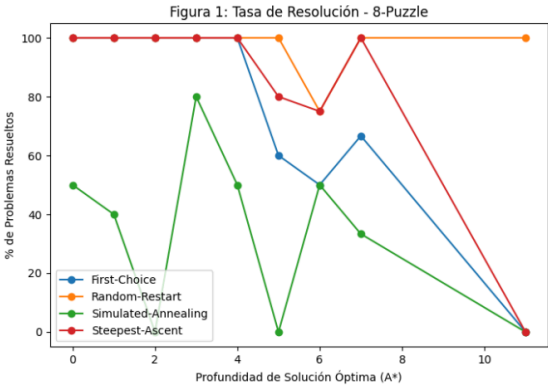
6. Results (data from your experiments or tests) and analysis

		SuccessRate	AvgSearchCost
Problem	Algorithm		
8-Puzzle	First-Choice	0.80	7.73
	Random-Restart	0.97	20.00
	Simulated-Annealing	0.37	1094.67
	Steepest-Ascent	0.90	11.83
8-Queens	First-Choice	0.10	102.57
	Random-Restart	0.80	1401.87
	Simulated-Annealing	0.40	28000.00
	Steepest-Ascent	0.23	238.93

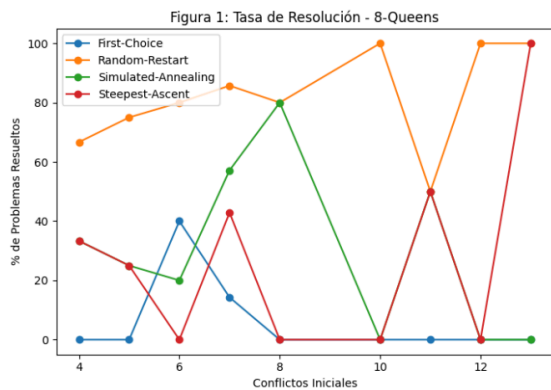
Tabla_1. muestra que el algoritmo Random-Restart obtiene la mayor tasa de éxito en ambos problemas, especialmente en 8-reinas, donde los óptimos locales

afectan significativamente a los métodos básicos de ascenso de colinas. Estos últimos funcionan bien en el 8-puzzle, pero fallan en problemas más complejos.

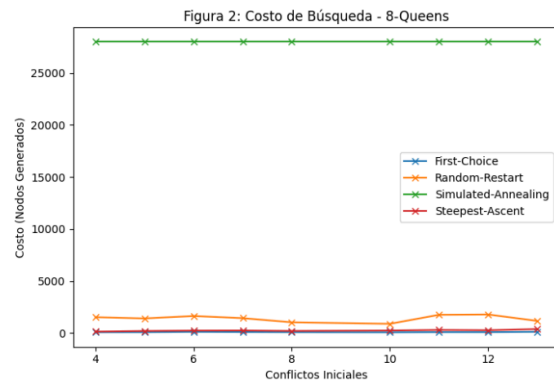
El recocido simulado presenta un desempeño intermedio en términos de éxito, pero con un costo computacional muy elevado, lo que sugiere una configuración no óptima. En conjunto, los resultados resaltan la importancia de incorporar mecanismos para escapar de los óptimos locales.



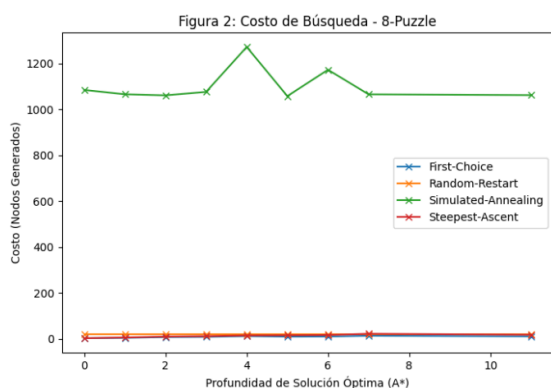
Figura_1 muestra que, a medida que aumenta la profundidad de la solución óptima, la tasa de éxito de los algoritmos disminuye. Random-Restart mantiene un rendimiento cercano al 100%, evidenciando su robustez. En contraste, Steepest-Ascent y First-Choice presentan una caída en su desempeño debido a su susceptibilidad a óptimos locales[1]. Simulated Annealing muestra un comportamiento inestable, lo que sugiere una configuración no óptima de sus parámetros.



Figura_2 muestra que, a medida que aumentan los conflictos iniciales, los algoritmos de ascenso de colinas presentan bajas tasas de éxito debido a la presencia de múltiples óptimos locales. Random-Restart mantiene un alto desempeño al reiniciar la búsqueda, mientras que Simulated Annealing presenta resultados intermedios pero inconsistentes.



Figura_4 muestra que Simulated Annealing tiene el mayor costo de búsqueda, debido a su exploración intensiva del espacio de estados. Los algoritmos de ascenso de colinas son mucho más eficientes, pero con menor tasa de éxito. Random-Restart logra un equilibrio entre costo y desempeño, justificando su mayor robustez frente a óptimos locales.



Figura_3 muestra que los algoritmos de ascenso de colinas tienen un costo de búsqueda bajo y constante, lo que evidencia su alta eficiencia. Random-Restart incrementa ligeramente el costo debido a los reinicios, pero sigue siendo eficiente. En contraste, Simulated Annealing presenta un costo muy elevado, indicando una exploración excesiva del espacio de estados.

Los resultados indican que el método de ascenso de colinas con reinicio aleatorio alcanza las tasas de éxito más altas en ambos problemas, llegando aproximadamente al 97 % para el rompecabezas de 8 piezas y al 80 % para el problema de las 8 reinas.

Los métodos de ascenso más pronunciado (steepest-ascent) y de primera elección (first-choice) presentan una alta eficiencia en términos de costo de búsqueda; sin embargo, muestran una fuerte sensibilidad a los óptimos locales, especialmente en el problema de las 8 reinas, donde el espacio de búsqueda es altamente multimodal.

Por su parte, el recocido simulado alcanza una tasa de éxito moderada, pero conlleva un costo computacional significativamente elevado, llegando hasta aproximadamente

28.000 nodos generados en el problema de las 8 reinas. Este comportamiento evidencia un claro compromiso entre robustez y eficiencia computacional.

Además, las figuras muestran que, a medida que aumenta la dificultad del problema, la tasa de éxito disminuye para los métodos de ascenso de colinas más simples, mientras que el método de reinicio aleatorio mantiene un rendimiento relativamente estable gracias a su capacidad para escapar de óptimos locales mediante múltiples reinicios.

7. Conclusions and lessons learned

El algoritmo de ascenso de colinas con reinicio aleatorio demostró ser el más eficaz en general debido a su capacidad para escapar de los óptimos locales. Los métodos de ascenso más pronunciado y de primera elección son eficientes, pero menos fiables. El recocido simulado requiere un ajuste preciso de parámetros para equilibrar la exploración y la eficiencia. [1]

Una lección clave aprendida es que la estructura del problema afecta significativamente el rendimiento del algoritmo, e incorporar mecanismos para escapar de óptimos locales es esencial para resolver problemas de búsqueda complejos.

8. Acknowledgements if any

9. Credits and references

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[2] N. G. Santiago and M. A. Jiménez, *Writing Formal Reports: An Approach for Engineering Students in 21st Century*, 3rd ed. Mayagüez, PR: UPRM, 2002. [Online]. Available: <https://ece.uprm.edu/~hunt/inel5326/ReporteFinal.pdf>. [Accessed: Mar. 28, 2024].

[3] P. Norvig and S. Russell, 'aima-python,' GitHub repository.

Youtube Presentation Link:

<https://youtu.be/IuCE10PI-vc>